



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

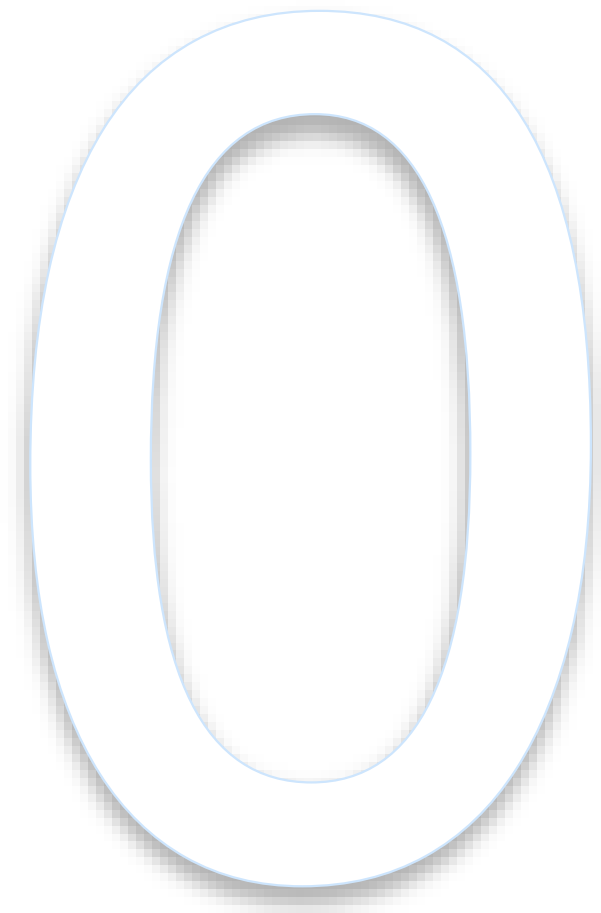
Удиртгал хичээл

Лекц 1

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- F.CSM101 хичээлийн танилцуулга
- ХИЙСВЭР МАШИН
 - Хийсвэр машины ойлголт
 - Интерпретатор
 - HW машин
 - Хэлний хэрэгжилт
 - Хийсвэр машин хэрэгжилт
 - Идеал тохиолдол
 - Бодит буюу Завсрын тохиолдол
 - Хийсвэр машины шатлал



#> F.CSM101 Программчлалын үндсэн аргууд

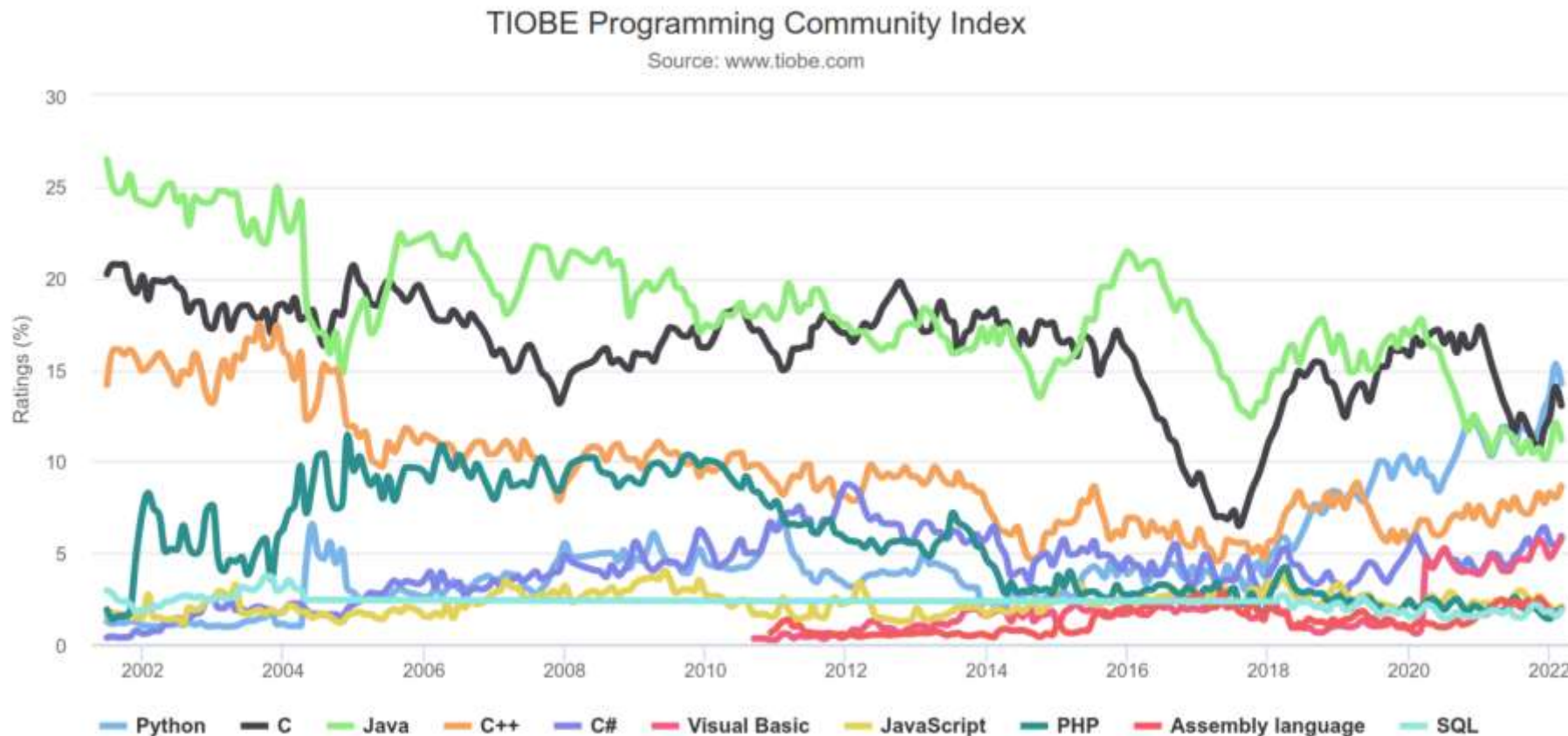
- **Программчлал**: Компьютерын ухааны судлаач, программ зохиогчдод зориулсан илэрхийллийн **үндсэн гол хэлбэр**.
 - “The limits of my language mean the limits of my world.” (Ludwig Wittgenstein)
 - Илэрхийлж болох санаа, алгоритмууд **Программчлалын хэл (PL)**-ээр хязгаарлагдана.

ХИЧЭЭЛИЙН ҮНДСЭН ЗОРИЛГО:

*Программчлалын хэл хэрхэн ажилладаг болохыг судлах замаар программчлалын суурь **парадигмуудыг** ойлгох, программчлалын хэл хооронд **хөрвөн ажиллах ур чадварыг** эзэмших*

Ямар нэг хэлийг дагнан судлахгүй байх???

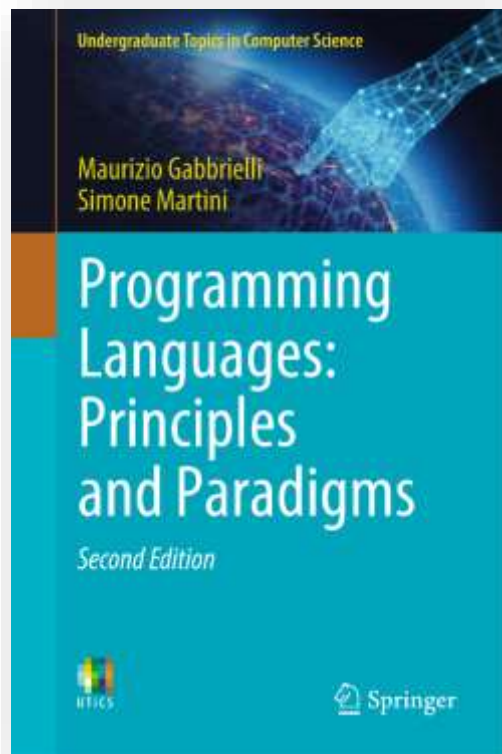
- **Иж бүрэн систем:** Янз бүрийн хэлээр бүтээгддэг
 - Жнь: Meta/Fb: Хэлний олон парадигмуудыг хамарсан хэлнүүдийн холимог
- **Шинэ хэл** байнга гарч ирдэг (хуучин хэлүүд хэрэглээнээс гардаг)



Gabbrielli M, Martini S

***Programming languages:
principles and paradigms, 2nd ed.***

Springer Nature; 2023

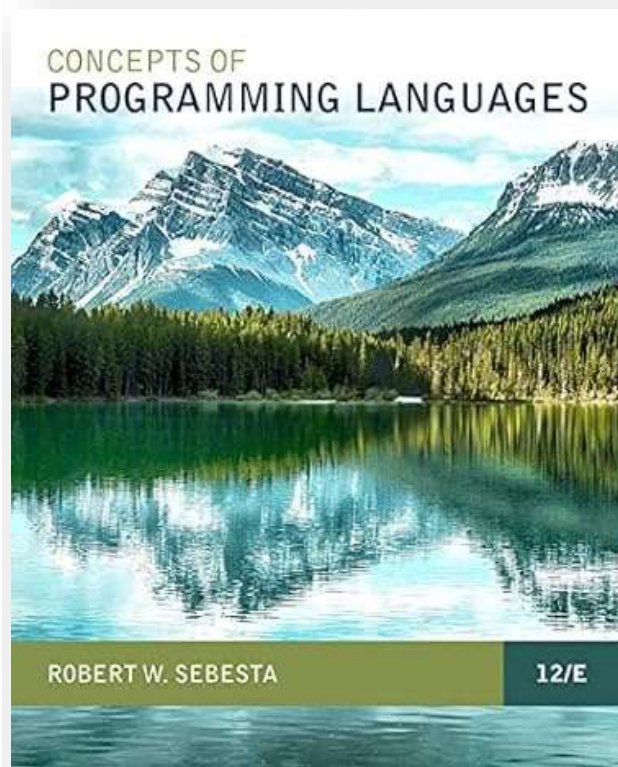


ҮНДСЭН СУРАХ БИЧИГ

Robert W. Sebesta

***Concepts of Programming
Languages, 12th ed;***

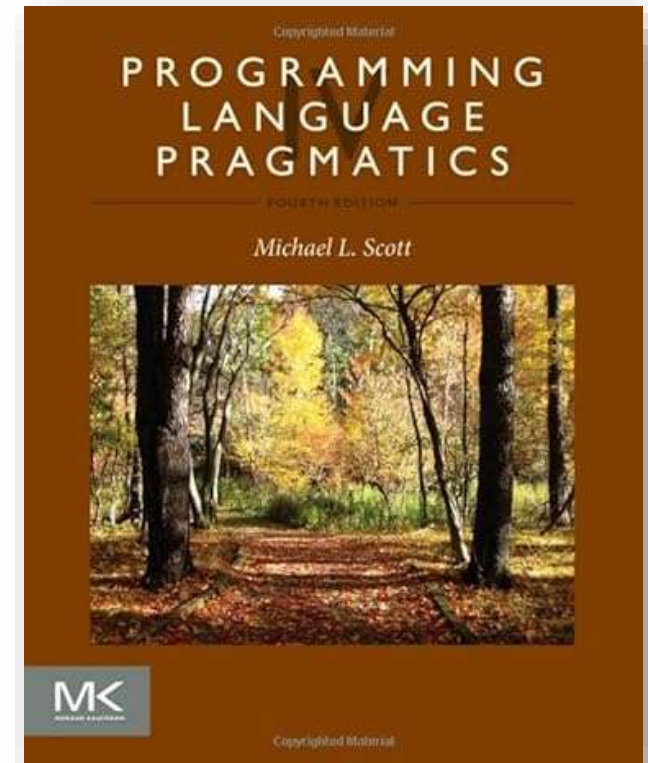
Pearson, 2019



Michael L. Scott

***Programming Language
Pragmatics. 4th ed.***

Morgan Kaufmann, 2015.



- Лабораторийн ажил – **15 × 2 = 30 оноо**

- Долоо хоног бүрт нь гаргана
- Танхимаар багшид хамгаална

- Бие даалтын ажил – **2 × 6 = 12 оноо**

- Үндсэн сурах бичгийн 1-7 бүлэгийн дасгалууд – 6 оноо.
- Үндсэн сурах бичгийн 8-15 бүлэгийн дасгалууд – 6 оноо.
- Latex дээр бичсэн тайлан илгээнэ

- Ирц идэвхи – **8 оноо**

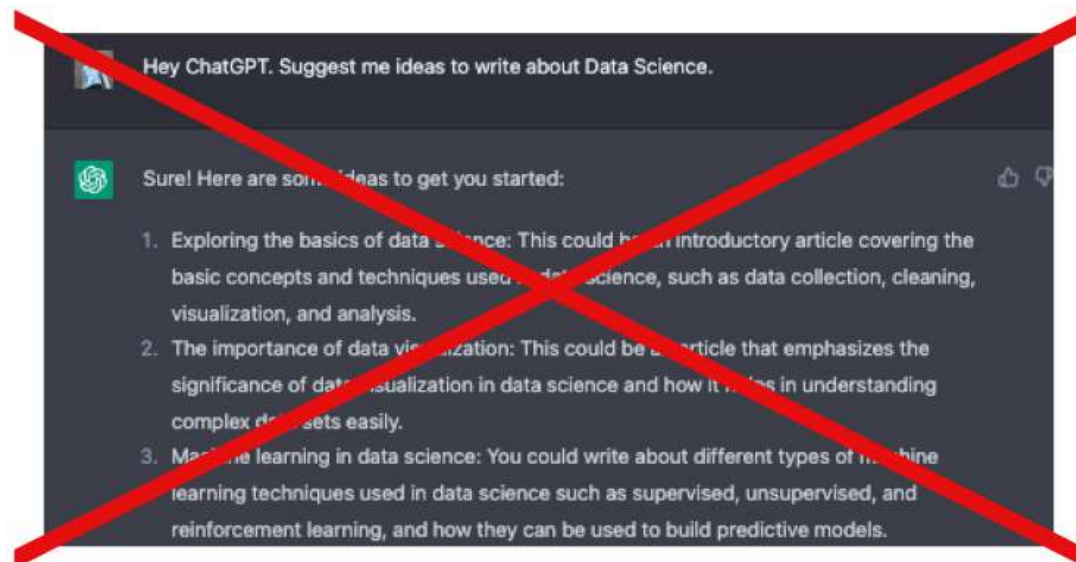
- Хичээлийн бүтэн ирц – 4 оноо
- Өөрийгөө сорих дасгалууд – 4 оноо
 - 2, 4, 6, 10, 12, 14-р долоо хоногт

- Бусад

- Улирлын шалгалт **30 оноо**
- Явцын сорил (7, 14-р) **20 оноо**

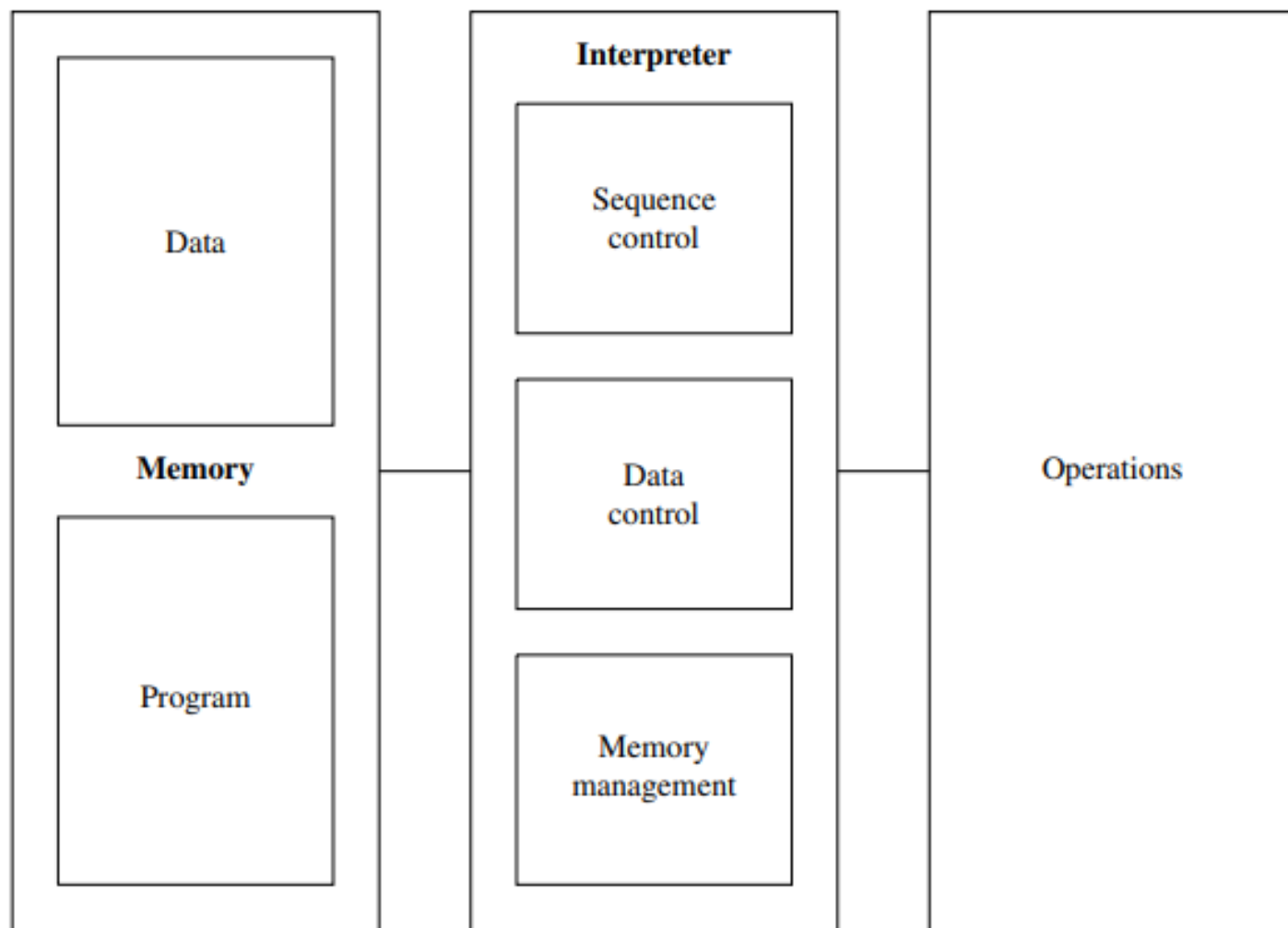
- *Wikipedia, Blogs эх сурвалж*
- *ChatGPT зэрэг AI түүл*
- *Шийдлээ бусдад дамжуулах*

Хориглоно !!!



**STOP DOING THIS
ON CHATGPT!**

Хийсвэр машин (Abstract Machines)



Хийсвэр машины бүтэц

Хийсвэрлэл

- Чухал бус нөхцлийг хасах
- Гол асуудалд анхаарах
- PL-үүд нийтлэг хэрэглэдэг

Хийсвэр машин

- Маш ерөнхий ойлголт
- PL-ний хэрэгжилтийг судлах
- Бүх тусгай хэрэгжилтийг нарийвчлах шаардлагагүй
- Шаталсан бүтцээр нь ПХ-ийн бүрэн системийг судлах

Сонгодог (generic) хийсвэр машин: *store* + *interpreter*

- Машин нь өөрсдийн ойлгохуйц формал алгоритмыг биелүүлдэг
- Хийсвэр машин - Физик машины хийсвэрлэл
- Алгоритм нь PL-ний бүтцийг (*заавар*) хэрэглэн формалчлагддаг
- Программчлалын хэл $\mathcal{L}$: Зааврын төгсгөлөг олонлог
- *Программ*: $\mathcal{L}$ хэлний заавруудын эрэмбэтэй, төгсгөлөг жагсаалт

ТОД 1.1 (*Хийсвэр машин*): $\mathcal{L}$ программчлалын хэл өгөгдсөн байг. $\mathcal{L}$ хэлээр бичигдсэн программыг хадгалах, гүйцэтгэх боломжтой аливаа өгөгдлийн бүтэц, алгоритмын багцыг $\mathcal{L}$ хэлний **хийсвэр машин** $\mathcal{M}_{\mathcal{L}}$ гэнэ.

Бүх хэлний интерпретатор: Үйлдлийн төрөл & “биелэлт”

1. Анхдагч (Primitive) дата-г боловсруулах үйлдлүүд;
2. Биелэлтийн дараалал хянах үйлдэл + дата бүтэц;
3. Дата дамжуулалтыг хянах үйлдэл + дата бүтэц;
4. Санах ойн удирдлагын үйлдэл + дата бүтэц.



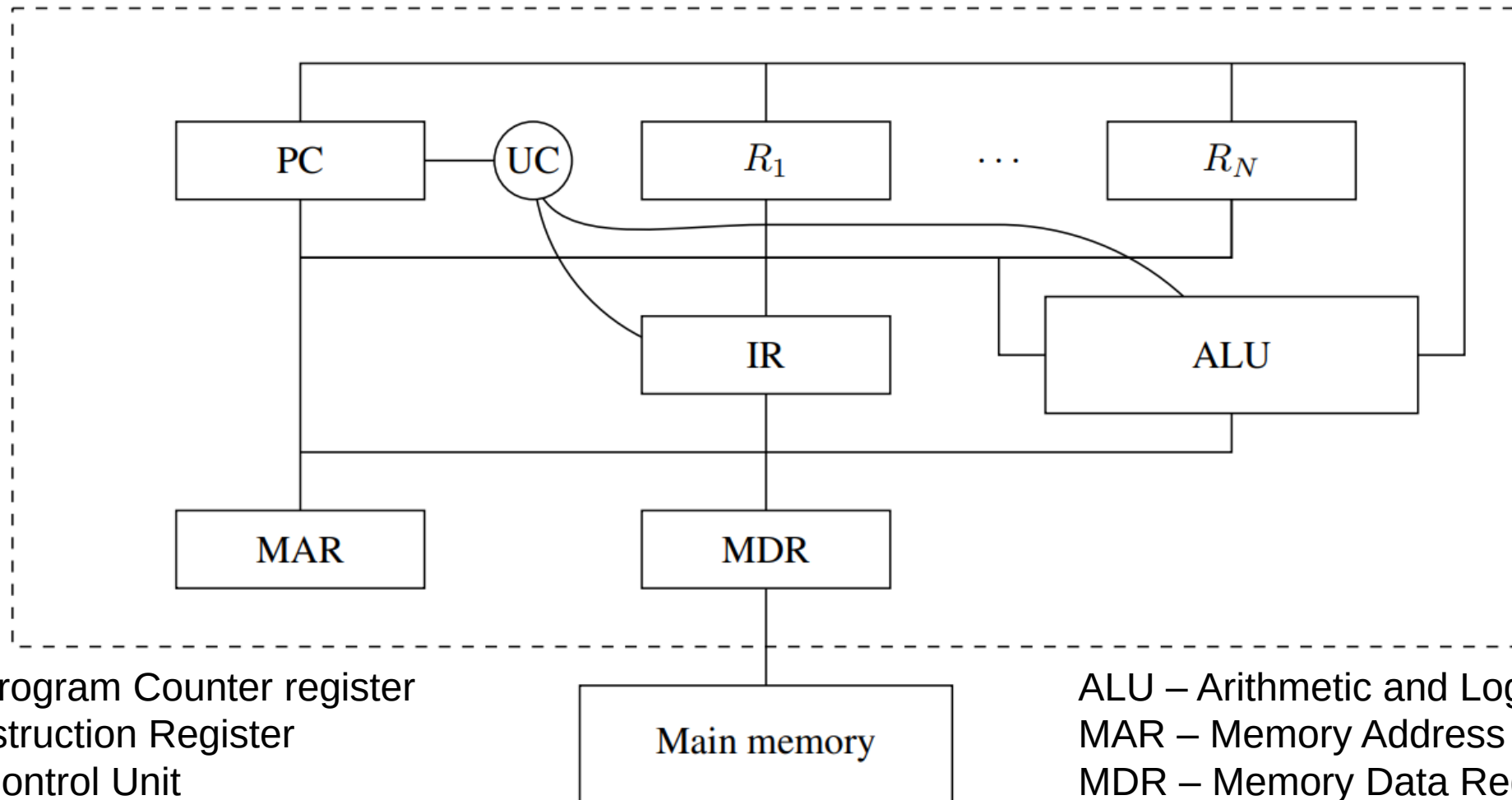
Сонгодог интерпретаторын биелэлт (execution) мөчлөг

- Доод түвшний хэл – Физик машинд илүү ойлгомжтой - Машины хэл
- Дээд түвшний хэл – Хүнд (программист) илүү ойлгомтой

ТОД 1.2 (*Машины хэл*): Хийсвэр машин $\mathcal{M}_{\mathcal{L}}$ өгөгдсөн бол түүний интерпретаторын "ойлгодог" $\mathcal{L}$ хэлийг $\mathcal{M}_{\mathcal{L}}$ -ийн *машины хэл* гэнэ.

- Физик шинж чанарт хараат байдлаар ажилладаг
- Симболик хэл - Ассемблэр хэл гэдэг
 - Хоёртын кодын оронд **ADD**, **MUL** гэх мэт тэмдэг хэрэглэдэг

Уламжлал физик машин: Логик гэйт болон Электрон компонентууд



- Хоёрдогч СО: Оптик/соронзон
- Үндсэн СО: тогтмол урттай дараалсан үг эсвэл үүр – 8, 32, 64 бит
- Кэш болон *регистр* – Central Processing Unit дотор байрладаг
- Бүх өгөгдлийг 2-тын хэлбэрээр хадгалдаг.
- Хэд хэдэн анхдагч “төрөл”-тэй: int, char, тогтмол урттай бит дараалал
- Зарим төрөл СО-н хэд хэдэн үг эзэлж болно. Жнь:
 - Floating-point нь нарийвчлалаасаа хамаарч нэг/хоёр үг
 - Alphanumeric нь 2-тын тоон дараалал хэрэглэнэ.
- Бүх өгөгдлийг бит дарааллаар дүрсэлдэг ч HW түвшинд анхдагч датаг HW үйлдлээр шууд удирдаж болох илүү нарийвчилсан *төрлүүд* байдаг.
 - *Урьдчилан тодорхойлсон төрөл* хэрэглэнэ.

Зааврын формат: OpCode Operand1 Operand2

- OpCode: HW анхдагч үйлдэл, давтагдахгүй код
- Operand: машины хадгалалтын бүтэц болон тэдгээрийн хаяглалтын горимоос хамааран операндуудыг олоход туслах утгууд
- Жишээ: а) ADD R5 , R0; ба b) ADD (R5), (R0)
 - а) R0, R5 утгуудыг нэмж R5-д хадгална
 - б) R0, R5 хаягтай үүрний утгуудыг нэмж R5 доторх хаяг руу хадгална
- Загварууд
 - CISC (*Complex Instruction Set Computer*): олон заавартай уламжлалт CISC (Complex Instruction Set Computer) процессорууд
 - RISC (*Reduced Instruction Set Computers*): цөөн заавартай, богино мөчлөгөөр пайплайн горимд хэрэгжүүлэх хангалттай энгийн архитектур

- НВ машины компонентууд
 1. **Үйлдэл – ALU**: Анхдагч өгөгдлийг боловсруулах үйлдлүүд (*Арифметик*: бүхэл тоо, хөвөгч цэгийн тоо, *boolean*: шилжилт, шалгалт гм).
 2. **Дарааллын хяналт – РС**: Дараагийн биелэх зааврын хаяг (+ +, үсрэлт)
 3. **Өгөгдөл дамжуулалт - CPU**: Санах ойтой холбогдсон CPU (MAR, MDR) регистрийг уншдаг. Хаяглалтын горимоос (*direct*, *indirect*) хамааран утгыг нь өөрчилдөг. CPU регистрт хандах, засах үйлдлүүдтэй.
 4. **СО-н боловсруулалт**: Энгийн архитектурт программыг ачаалаад эхлүүлдэг. Модерн архитектурт хурд нэмэх үүднээс шаталсан СО (кэш гм), түр зогсоох механизм ашигладаг (НВ дэмжлэг, Регистрийн оронд *push*, *pop* стэйк).
- Сонгодог интерпретатортай аналог чанартай: *fetch-decode-execute*
 - *fetch*: Санах ойгоос дараагийн ажиллах зааврыг олж авчирна
 - *decode*: Зааврыг задлан уншиж хаяглалт болон операндыг мэднэ
 - *execute*: Анхдагч НВ үйлдлийг гүйцэтгэнэ. Жнь ALU хэрэглэнэ.

- $\mathcal{M}_{\mathcal{L}}$ хийсвэр машины хувьд тодорхойлолт ёсоор
 - $\mathcal{L}$ хэл дээр бичигдсэн программыг биелүүлэх тул **машин хэл** нь $\mathcal{L}$ байна.
- Эсрэгээрээ машин хэл нь $\mathcal{L}$ байх хэд хэдэн хийсвэр машин байна.
 - Интерпретаторын хэрэгжүүлэлт, дата бүтцээрээ ялгаатай.
 - Интерпертат хийж буй $\mathcal{L}$ хэл дээрээ нэгддэг

$\mathcal{L}$ хэл **хэрэгжих**:

Машин хэл нь $\mathcal{L}$ байх хийсвэр машин биелэгдэх.

- Бодит байдал дээр $\mathcal{L}$ -ийн зааврыг биелүүлдэг $\mathcal{M}_{\mathcal{L}}$ -ийн төрөл бүрийн физик төхөөрөмж байх боломжтой:
 - механик, электроник, биологийн г.м
- Хэлбэр
 - Ил (explicit) – Шууд физик хэрэгжүүлэлт.
 - Далд (implicit) – Физик төхөөрөмж болон $\mathcal{M}_{\mathcal{L}}$ хооронд завсрын.
- Ангилал:
 - Шууд техник хангамж хэрэгжүүлэлт
 - ПХ-ийн **симуляц**
 - Үйлдвэрийн ПХ-ийн (firmware) симуляц (**эмуляц**)

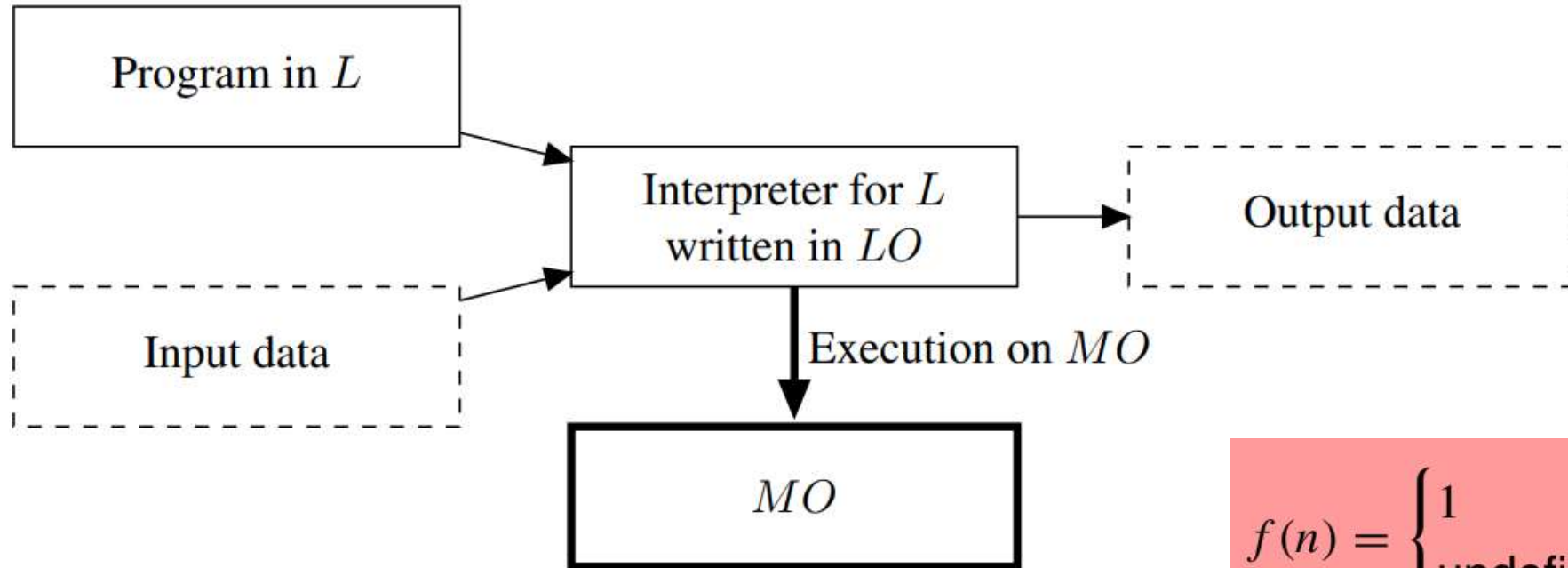
- Зарчмын хувьд тодорхой, маш энгийн ойлголт
 - СО, арифметик/логик хэлхээ, дамжуулалтын шугам зэрэг физик төхөөрөмжийг ашиглан $\mathcal{L}$ машины хэлтэй байхаар хэрэгжүүлэх
 - Ингэхийн тулд хийсвэр машины дата бүтэц + алгоритмуудыг HW-д хэрэгжүүлэхэд хангалттай.
- *Давуу тал:* Шууд төхөөрөмжийн гүйцэтгэл учраас маш хурдан
 - Онцгой хурдны гүйцэтгэлтэй HW-т шууд чиглэсэн хэрэглээнд “тусгайлан зориулсан” хэлийг хэрэгжүүлэх боломжтой
- *Дутагдалтай тал:* Дээд түвшний сонгодог хэл нь электрон хэлхээний хэт энгийн функцээс хол
 - Физик машины дизайн илүү төвөгтэй болно. Практикт ийм машиныг хэрэгжүүлсэн л бол их өртөгтэй учир өөрчилдөггүй
 - Доод түвшний программчлалын хэлний хувьд л, тухайн физик төхөөрөмжийн хийж чадах үйлдэлд илүү ойрхон байхаар бүтээдэг

- ПХ-ийн симуляц
- $\mathcal{M}_{\mathcal{L}}$ машинаар өөр нэг $\mathcal{L}'$ хэл дээр бичигдсэн программыг ажиллуулах дата бүтэц + алгоритм бүхий хийсвэр машиныг хэрэгжүүлэх.
 - $\mathcal{L}'$ хэлээр бичигдсэн программыг $\mathcal{M}_{\mathcal{L}}$ машины функционалыг дуурайн $\mathcal{L}$ хэлний бүтцээр хөрвүүлдэг $\mathcal{M}'_{\mathcal{L}}$ машины тусламжтайгаар $\mathcal{M}_{\mathcal{L}}$ машиныг хэрэгжүүлж чадна.
 - Хэрэгжилтийн тусдаа түвшин нэмэгдэж байна.

Микропрограмм: Үндсэн СО-д биш зөвхөн уншигддаг тусгай СО-д хадгалагддаг, физик машин дээр дээд хурдаар ажилладаг, маш доод түвшинд анхдагч үйлдлийг гүйцэтгэдэг

- Үйлдвэрийн ПХ-ийн симуляц буюу Эмуляц
 - *Микропрограммчлал* дэмждэг физик машины хувьд боломжтой шийдэл
 - $\mathcal{M}_L$ машины дата бүтэц + алгоритмуудын *микрокод*-д зориулсан симуляц
- *Давуу тал:* Дээд түвшний хэлний ПХ-ийн оронд микропрограмм байдаг.
 - HW шийдлээс хурдангүй ч ПХ-ийн симуляцаас илүү
- *Дутагдалтай тал:*
 - ПХ-ын симуляцын түвшний хэлээр бичигдсэн программтай харьцуулахад Микрокодны засвар, өөрчлөлт илүү төвөгтэй

- $\mathcal{M}_{\mathcal{L}}$ хийсвэр машинд шаарддагдах сонгодог $\mathcal{L}$ хэлний хувьд:
 - $\mathcal{M}_{\mathcal{L}}$ -ийн HW-ийн шууд хэрэгжилтийг орхие
 - Хийсвэр машин $\mathcal{M}_{O_{\mathcal{L}O}}$ (*Хост машин* гэнэ) байгаа гэж үзье:
 - $\mathcal{L}O$ машин хэлэнд зориулан хэрэгжүүлсэн (яаж гэдэг нь чухал биш)
- $\mathcal{M}_{O_{\mathcal{L}O}}$ дээрх $\mathcal{L}$ хэлний хэрэгжилт: $\mathcal{L} \rightarrow \mathcal{L}O$ "хөрвүүлэлт (translation)"
 1. *Интерпретац дагнасан хэрэгжилт* – ил (explicit)
 2. *Компиляц дагнасан хэрэгжилт* – далд (implicit)

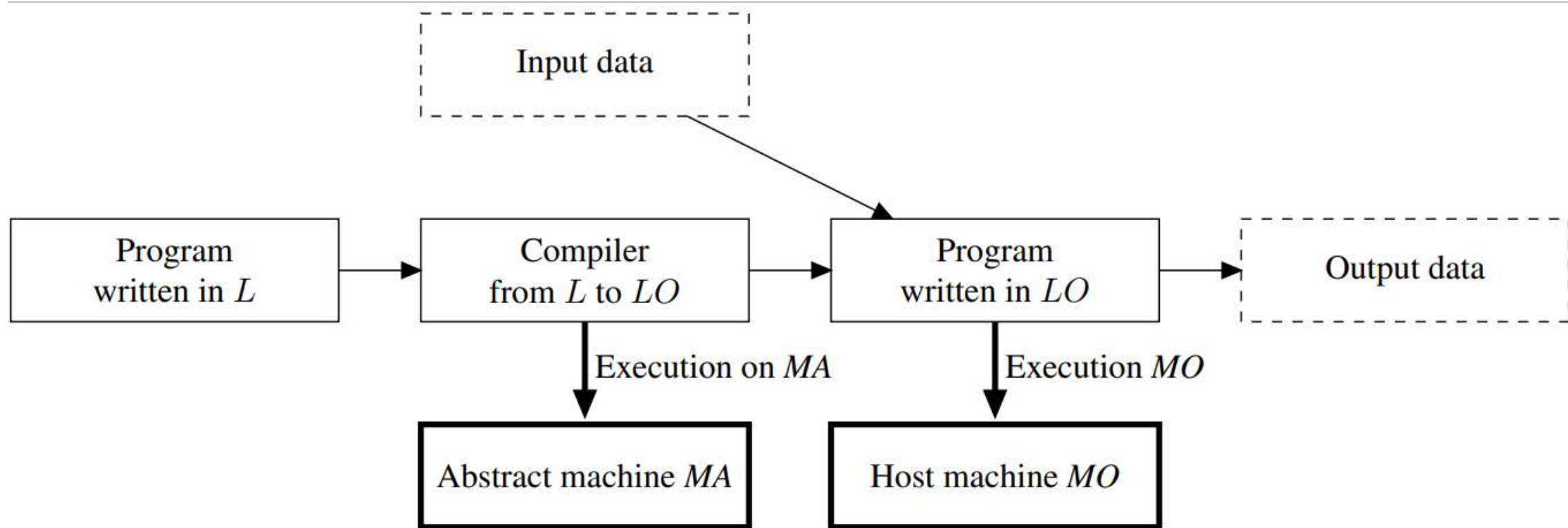


Хагас функц

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \\ \text{undefined} & \text{otherwise} \end{cases}$$

ТОД 1.3 (Интерпретатор): $\mathcal{L}_0$ хэл дээр бичигдсэн $\mathcal{L}$ хэлний интерпретатор бол дараах хагас функцийг хэрэгжүүлдэг **программ** :

$$\mathcal{I}_{\mathcal{L}_0}^{\mathcal{L}}(\mathcal{P}^{\mathcal{L}}, \text{Input}) = \mathcal{P}^{\mathcal{L}}(\text{Input}) \quad \text{байх} \quad \mathcal{I}_{\mathcal{L}_0}^{\mathcal{L}} : (\text{Prog}^{\mathcal{L}} \times \mathcal{D}) \rightarrow \mathcal{D}$$



ТОД 1.4 (Компилятор): $\mathcal{L}$ -ээс $\mathcal{L}_o$ -рүү компилятор нь дараах функцийг хэрэгжүүлдэг **программ**: $\mathcal{C}_{\mathcal{L}, \mathcal{L}_o} : \text{Prog}^{\mathcal{L}} \rightarrow \text{Prog}^{\mathcal{L}_o}$.

Хэрэв $\mathcal{C}_{\mathcal{L}, \mathcal{L}_o}(\mathcal{P}^{\mathcal{L}}) = \mathcal{P}^{\mathcal{L}_o}$ байдаг бол $\mathcal{P}^{\mathcal{L}}(\text{Input}) = \mathcal{P}^{\mathcal{L}_o}(\text{Input})$ байна.

```
P1: for (I = 1, I<=n, I=I+1) C;
```

Интерпретат - Ажиллах үед командыг шууд хөрвүүлээд биелүүлнэ

- for үйлдэл машин хэлэндээ байхгүй тохиолдол давталт болгоно
- Хөрвүүлээд хадгалахгүй тул for үйлдэл тааралдах бүрт хөрвүүлнэ

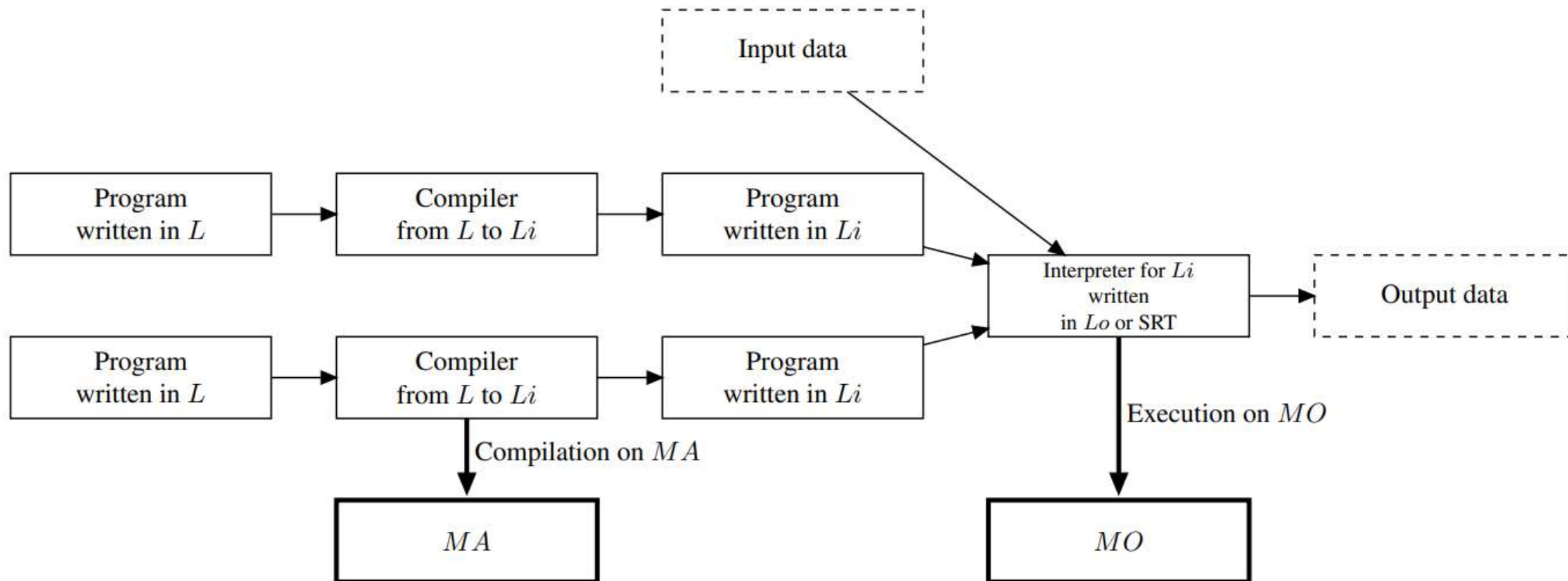
Компиляц: Ажиллах үедээ машин хэлний P2-ийг бүтнээр нь үүсгэнэ

- for Команд давтагдахад дахин хөрвүүлэхгүй

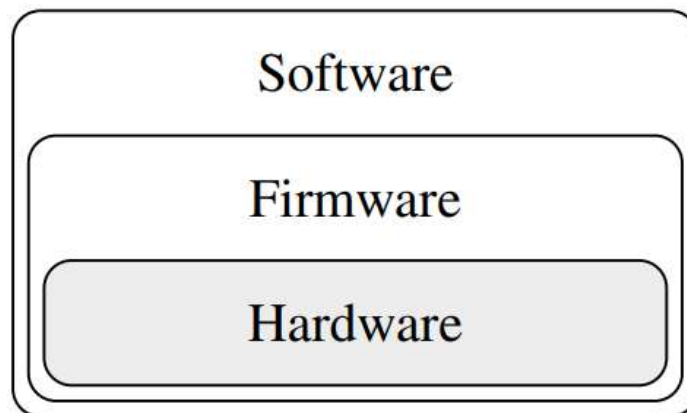
```
P2 :  
    R1 = 1  
    R2 = n  
L1: if R1 > R2 then goto L2  
    translation of C  
    ...  
    R1 = R1 + 1  
    goto L1  
L2: ...
```


- Интерпретат
 - хөрвүүлгийн үе байхгүй – үр ашиг багатай
 - Код тайлах хугацаа – кодыг тааралдах бүрт давтан хийнэ
 - Ажиллах үедээ хөрвүүлнэ – Дебаг хийхэд онцгай ач холбогдолтой
 - Хөрвүүлж шинэ программ гаргадаггүй – Санах ойд хэмнэлттэй
- Компилят
 - Програмыг хөрвүүлж шинийг үүсгэнэ – кодыг давтан тайлахгүй
 - Программ биелэлт нь тусдаа – ажиллах үед зааврын код тайлахгүй
 - Эх программын бүтцийн бүх мэдээллийг орхидог – гол сул тал
 - Компиляц хийсэн кодыг ашигласан бол – Дебаг хийхэд хүндрэлтэй

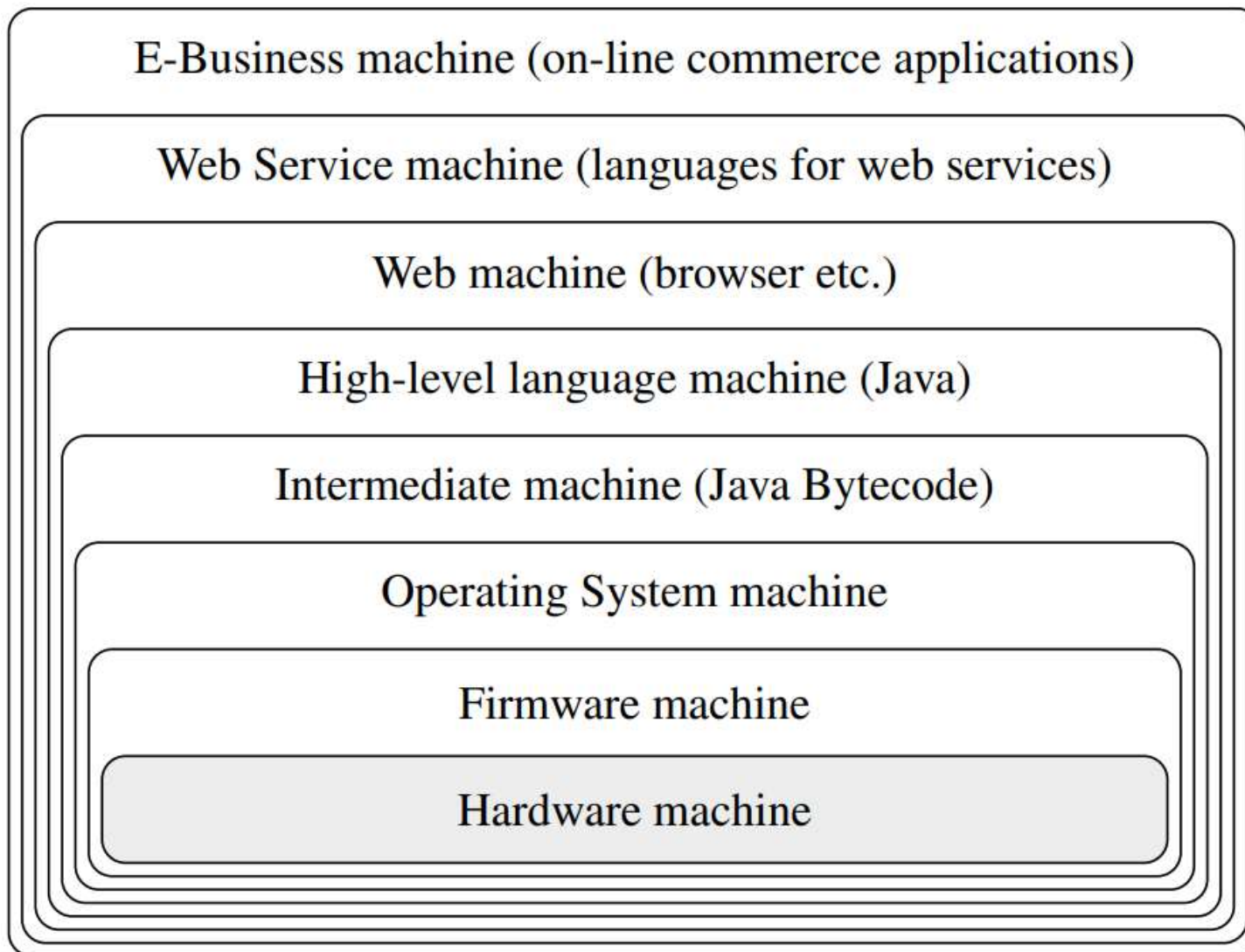
- Бодит хэлний хэрэгжилтэд өмнөх хоёр хэрэглээ бараг байнга байдаг
 - Интерпретатор бүр программын гаднахаасаа дотор тал нь үргэлж өөр
 - Компиляц бүр тусгай комплекс бүтцийг симуляц хийдэг
 - Жнь: Оролт/гаралтыг хэрэгжүүлэхэд их заавар шаардах тул ажиллах үед нь симуляц хийх боломжтой программын дуудалт болгох
- Засврын түвшний болон хост түвшний хоорондох зайнаас хамааран өөр өөр төрлийн хэрэгжилттэй.
 1. $\mathcal{M}_{\mathcal{L}} = \mathcal{M}i_{\mathcal{L}i}$: интерпретац дагнсан хэрэгжилт.
 2. $\mathcal{M}_{\mathcal{L}} \neq \mathcal{M}i_{\mathcal{L}i} \neq \mathcal{M}o_{\mathcal{L}o}$.
 - a) Хэрэв завсрын машины интерпретатор нь $\mathcal{M}o_{\mathcal{L}o}$ -ийнхоос эрс ялгаатай бол интерпретат төрлийн хэрэгжилттэй
 - b) Хэрэв завсрын машины интерпретатор нь $\mathcal{M}o_{\mathcal{L}o}$ -той үндсэндээ ижил бол (зарим функцийг нь дуудалт болгосон) компилят төрлийн хэрэгжилттэй
 3. $\mathcal{M}i_{\mathcal{L}i} = \mathcal{M}o_{\mathcal{L}o}$, бид компиляц дагнасан хэрэгжилттэй.



- Энэ схемийг дурын түвшин хүртэл шатлан өргөтгөх боломжтой.
- Машин бүр доор байгаа түвшний функцийг ашиглаж, дээрх түвшинд өөрийн шинэ функцийг гаргаж өгдөг.
- $\mathcal{L}$ хэлээр P программ нь интерфэйсээрээ дамжуулан хэрэглэгчийг хангадаг функционалаас бүрдсэн шинэ $\mathcal{L}_P$ хэл болно гэсэн үг (шинэ хийсвэр машиныг хэрэгжүүлж байна).



Дээд түвшний програмчлалын хэлийг хэрэгжүүлсэн микропрограммчлалтай компьютер



ПХ-ийн системийг хийсвэр машинуудын давхаргын хувьд зохион байгуулах нь

- Давхаргын үйл ажиллагааны дотоод хэрэгжилтийг өөрчлөх нь бусад давхаргад ямар ч нөлөө үзүүлэхгүй
- Төрөл бүрийн давхаргуудын бие даасан байдлыг хангах боломжийг олгодог.



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА